



Chapter 7

Data Inspection Features

There are three broad kinds of data domains: cardinal, ordinal, and nominal. The first project in this chapter will guide you through the inspection of cardinal data; values like weights, measures, and durations where the data is continuous, as well as counts where the data is discrete. The second project will guide reasoners through the inspection of ordinal data involving things like dates, where order matters, but the data isn't a proper measurement; it's more of a code or designator. The nominal data is a code that happens to use digits but doesn't represent numeric values. The third project will cover the more complex case of matching keys between separate data sources.

An inspection notebook is required when looking at new data. It's a great place to keep notes and lessons learned. It's helpful when diagnosing problems that arise in a more mature analysis pipeline.

This chapter will cover a number of skills related to data inspection techniques:

- Essential notebook data inspection features using Python expressions, extended from the previous chapter.
- The `statistics` module for examining cardinal data.
- The `collections.Counter` class for examining ordinal and nominal data.
- Some additional `collections.Counter` for matching primary and foreign keys.

For the Ancombe's Quartet example data set used in *Chapters 3, 4, and 5*, both of the attribute values are cardinal data. It's a helpful data set for some of the inspections, but we'll need to look at some other data sets for later projects in this chapter. We'll start by looking at some inspection techniques for cardinal data. Readers who are focused on other data sets will need to discern which attributes represent cardinal data.

7.1 Project 2.2: Validating cardinal domains — measures, counts, and durations

A great deal of data is cardinal in nature. Cardinal numbers are used to count things, like elements of a set. The concept can be generalized to include real numbers representing a weight or a measure.

A very interesting data set is available here:

<https://www.kaggle.com/datasets/rtatman/iris-dataset-json-version>.

This contains samples with numerous measurements of the pistils and stamen of different species of flowers. The measurements are identifiable because the unit, mm, is provided.

Another interesting data set is available here:

<https://datahub.io/core/co2-ppm>. This contains data with measurements of CO2 levels measured with units of ppm, parts per million.

We need to distinguish counts and measures from numbers that are only used to rank or order things, which are called ordinal numbers. Also, number-like data is sometimes only a code. US postal codes, for example, are merely strings of digits; they aren't proper numeric values. We'll look at these numeric values in [***Project 2.3: Validating text and codes — nominal data and ordinal numbers***](#).

Since this is an inspection notebook, the primary purpose is only to understand the range of values for cardinal data. A deeper analysis will come later. For now, we want a notebook that demonstrates the data is complete and consistent, and can be used for further processing.

In the event an enterprise is using data contracts, this notebook will demonstrate compliance with the data contract. With data contracts, the focus may shift slightly from showing “some data that is not usable” to showing “data found to be non-compliant with the contract.” In cases where the contract is inadequate for the analytical consumer, the notebook may shift further to show “compliant data that’s not useful.”

We’ll start with a description of the kinds of cells to add to an inspection notebook. After that, we’ll about the architectural approach and wrap up with a detailed list of deliverables.

7.1.1 Description

This project’s intent is to inspect raw data to understand if it is actually cardinal data. In some cases, floating-point values may have been used to represent nominal data; the data appears to be a measurement but is actually a code.

Spreadsheet software tends to transform all data into floating-point numbers; many data items may look like cardinal data.

One example is US Postal Codes, which are strings of digits, but may be transformed into numeric values by a spreadsheet.

Another example is bank account numbers, which — while very long — can be converted into floating-point numbers. A floating-point value uses 8 bytes of storage, but will comfortably represent about 15 decimal digits. While this is a net saving in storage, it is a potential confusion of data types and there is a (small) possibility of having an account number altered by floating-point truncation rules.

The user experience is a Jupyter Lab notebook that can be used to examine the data, show some essential features of the raw data values, and confirm that the data really does appear to be cardinal.

There are several common sub-varieties of cardinal data:

- Counts; represented by integer values.
- Currency and other money-related values. These are often decimal values, and the `float` type is likely to be a bad idea.
- Duration values. These are often measured in days, hours, and minutes, but represent a time interval or a “delta” applied to a point in time. These can be normalized to seconds or days and represented by a float value.
- More general measures are not in any of the previous categories. These are often represented by floating-point values.

What’s important for this project is to have an overview of the data. Later projects will look at cleaning and converting the data for further use. This notebook is only designed to preview and inspect the data.

We’ll look at general measures first since the principles apply to counts and durations. Currency, as well as duration, values are a bit more complicated and we’ll look at them separately. Date-time stamps are something we’ll look at in the next project since they’re often thought of as ordinal data, not cardinal.

7.1.2 Approach

This project is based on the initial inspection notebook from [*Chapter 6, Project 2.1: Data Inspection Notebook*](#). Some of the essential cell content will be reused in this notebook. We’ll add components to the components shown in the earlier chapter – specifically, the `samples_iter()` function to iterate over samples in an open file. This feature will be central to working with the raw data.

In the previous chapter, we suggested avoiding conversion functions. When starting down the path of inspecting data, it’s best to assume nothing and look at the text values first.

There are some common patterns in the source data values:

- The values appear to be all numeric values. The `int()` or `float()` function works on all of the values. There are two sub-cases here:
 - All of the values seem to be proper counts or measures in some expected range. This is ideal.
 - A few “outlier” values are present. These are values that seem to be outside the expected range of values.
- Some of the values are not valid numbers. They may be empty strings, or a code line “NULL”, “None”, or “N/A”.

Numeric outlier values can be measurement errors or an interesting phenomenon buried in the data. Outlier values can also be numeric code values indicating a known missing or otherwise unusable value for a sample. In the example of the CO2 data, there are outlier values of -99.99 parts per million, which encode a specific kind of missing data situation.

Many data sets will be accompanied by metadata to explain the domain of values, including non-numeric values, as well as the numeric codes in use. Some enterprise data sources will not have complete or carefully explained metadata. This means an analyst needs to ask questions to locate the root cause for non-numeric values or special codes that appear in cardinal data.

The first question — *are all the values numeric?* — can be handled with code like the following:

```
from collections import defaultdict
from collections.abc import Iterable, Callable
from typing import TypeAlias

Conversion: TypeAlias = Callable[[str], int | float]

def non_numeric(test: Conversion, samples: Iterable[str]) -> dict[str, int]:
    bad_data = defaultdict(int)
    for s in samples:
        try:
            test(s)
        except ValueError:
```

```
        bad_data[s] += 1
    return bad_data
```

The idea is to apply a conversion function, commonly `int()` or `float()`, but `decimal.Decimal()` may be useful for currency data or other data with a fixed number of decimal places. If the conversion function fails, the exceptional data is preserved in a mapping showing the counts.

You're encouraged to try this with a sequence of strings like the following:

```
data = ["2", "3.14", "42", "Nope", None, ""]
non_numeroc(int, data)
```

This kind of test case will let you see how this function works with good (and bad) data. It can help to transform the test case into a docstring, and include it in the function definition.

If the result of the `non_numeric()` function is an empty dictionary, then the lack of non-numeric data means all of the data is numeric.

The test function is provided first to follow the pattern of higher-order functions like `map()` and `filter()`.

A variation on this function can be used as a numeric filter to pass the numeric values and reject the non-numeric values. This would look like the following:

```
from collections.abc import Iterable, Iterator, Callable
from typing import TypeVar

Num = TypeVar('Num')

def numeric_filter(
    conversion: Callable[[str], Num],
    samples: Iterable[str]
) -> Iterator[Num]:
    for s in samples:
```

```
try:
    yield conversion(s)
except ValueError:
    pass
```

This function will silently reject the values that cannot be converted. The net effect of omitting the data is to create a NULL that does not participate in further computations. An alternative may be to replace invalid values with default values. An even more complicated choice is to interpolate a replacement value using adjacent values. Omitting samples may have a significant impact on the statistical measures used in later stages of processing. This `numeric_filter()` function permits the use of other statistical functions to locate outliers.

For data with good documentation or a data contract, outlier values like -99.99 are easy to spot. For data without good documentation, a statistical test might be more appropriate. See

<https://www.itl.nist.gov/div898/handbook/eda/section3/eda35h.htm>

for details on approaches to locating outliers.

One approach suitable for small data sets is to use a median-based Z-score. We'll dive into an algorithm that is built on a number of common statistical measures. This will involve computing the median using a function available in the built-in `statistics` package.

For more information on basic statistics for data analytics, see *Statistics for Data Science*.

<https://www.packtpub.com/product/statistics-for-data-science/9781788290678>.

The conventional Z-score for a sample, Z_i , is based on the mean, $\bar{Y}$, and the standard deviation, σ_Y . It's computed as $Z_i = \frac{Y_i - \bar{Y}}{\sigma_Y}$. It measures how many standard deviations a value lies from the mean. Parallel with this is the idea of a median-based Z-score, M_i . The median-based Z-score uses the median, $\tilde{Y}$, and the median absolute deviation, MAD_Y .

This is computed as $M_i = \frac{Y_i - \tilde{Y}}{\text{MAD}_Y}$. This measures how many “MAD” units a value lies from the median of the samples.

The MAD is the median of the absolute values of deviations from the median. It requires computing an overall median, $\tilde{Y}$, then computing all the deviations from the overall median, $Y_i - \tilde{Y}$. From this sequence of deviations from the median, the median value is selected to locate a central value for all of the median absolute deviations. This is computed as $\text{MAD}_Y = \text{median}(|Y_i - \tilde{Y}|)$.

The filter based on M_i looks for any absolute value of the deviation from MAD_Y that’s greater than 3.5, $|M_i| > 3.5$. These samples are possible outliers because their absolute deviation from the median is suspiciously large.

To be complete, here’s a cell to read the source data:

```
with series_4_path.open() as source_file:
    series_4_data = list(samples_iter(source_file))
```

This can be followed with a cell to compute the median and the median absolute deviation. The median computation can be done with the `statistics` module. The deviations can then be computed with a generator, and the median absolute deviation computed from the generator. The cell looks like the following:

```
from statistics import median

y_text = (s['y'] for s in series_4_data)
y = list(numeric_filter(float, y_text))
m_y = median(y)
mad_y = median(abs(y_i - m_y) for y_i in y)
outliers_y = list(
    filter(lambda m_i: m_i > 3.5, ((y_i - m_y)/mad_y for y_i in y))
)
```

The value of `y_text` is a generator that will extract the values mapped to the `'y'` key in each of the raw data samples in the NDJSON file. From

these text values, the value of `y` is computed by applying the `numeric_filter()` function.

It's sometimes helpful to show that `len(y) == len(y_text)` to demonstrate that all values are numeric. In some data sets, the presence of non-numeric data might be a warning that there are deeper problems.

The value of `m_y` is the median of the `y` values. This is used to compute the MAD value as the median of the absolute deviations from the median. This median absolute deviation provides an expected range around the median.

The `outliers_y` computation uses a generator expression to compute the median-based Z-score, and then keep only those scores that are more than 3.5 MADs from the median.

The data in Series IV of Anscombe's Quartet seems to suffer from an even more complicated outlier problem. While the "x" attribute has a potential outlier, the "y" attribute's MAD is zero. This means more than half the "y" attribute values are the same. This single value is the median, and the difference from the median will be zero for most of the samples.

This anomaly would become an interesting part of the notebook.

Dealing with currency and related values

Most currencies around the world use a fixed number of decimal places. The United States, for example, uses exactly two decimal places for money. These are decimal values; the `float` type is almost always the wrong type for these values.

Python has a `decimal` module with a `Decimal` type, which must be used for currency.

Do not use `float` for currency or anything used in currency-related computations.

Tax rates, discount rates, interest rates, and other money-related fields are also decimal values. They're often used with currency values, and computations must be done using decimal arithmetic rules.

When we multiply `Decimal` values together, the results may have additional digits to the right of the decimal place. This requires applying rounding rules to determine how to round or truncate the extra digits. The rules are essential to getting the correct results. The `float` type `round()` function may not do this properly. The `decimal` module includes a wide variety of rounding and truncating algorithms.

Consider an item with a price of \$12.99 in a locale that charges a sales tax of 6.25% on each purchase. This is not a tax amount of \$0.811875. The tax amount must be rounded; there are many, many rounding rules in common use by accountants. It's essential to know which rule is required to compute the correct result.

Because the underlying assumption behind currency is decimal computation, the `float` should never be used for currency amounts.

This can be a problem when spreadsheet data is involved. Spreadsheet software generally uses `float` values with complex formatting rules to produce correct-looking answers. This can lead to odd-looking values in a CSV extract like 12.999999997 for an attribute that should have currency values.

Additionally, currency may be decorated with currency symbols like \$, £, or €. There may also be separator characters thrown in, depending on the locale. For the US locale, this can mean stray "," characters may be present in large numbers.

The ways currency values may have text decoration suggest the conversion function used by a `non_numeric()` or `numeric_filter()` function will have to be somewhat more sophisticated than the simple use of the `Decimal` class.

Because of these kinds of anomalies, data inspection is a critical step in data acquisition and analysis.

Dealing with intervals or durations

Some data will include duration data in the form "12:34", meaning 12 hours and 34 minutes. This looks exactly like a time of day. In some cases, it might have the form 12h 34m, which is a bit easier to parse. Without metadata to explain if an attribute is a duration or a time of day, this may be impossible to understand.

For durations, it's helpful to represent the values as a single, common time unit. Seconds are a popular choice. Days are another common choice.

We can create a cell with a given string, for example:

```
time_text = "12:34"
```

Given this string, we can create a cell to compute the duration in seconds as follows:

```
import re

m = re.match(r"(\d+):(\d+)", time_text)
h, m = map(int, m.groups())
sec = (h*60 + m) * 60
sec
```

This will compute a duration, `sec`, of 45,240 seconds from the source time as text, `time_text`. The final expression `sec` in a Jupyter notebook cell will display this variable's value to confirm the computation worked. This cardinal value computation works out elegantly.

For formatting purposes, the inverse computation can be helpful. A floating-point value like 45,240 can be converted back into a sequence of integers, like (12, 34, 0), which can be formatted as "12:34" or "12h 34m 0s".

It might look like this:

```
h_m, s = divmod(sec, 60)
h, m = divmod(h_m, 60)
text = f"{h:02d}:{m:02d}"
text
```

This will produce the string `12:34` from the value of seconds given in the `sec` variable. The final expression `text` in a cell will display the computed value to help confirm the cell works.

It's important to normalize duration strings and complex-looking times into a single float value.

Now that we've looked at some of the tricky cardinal data fields, we can look at the notebook as a whole. In the next section, we'll look at refactoring the notebook to create a useful module.

Extract notebook functions

The computation of ordinary Z-scores and median-based Z-scores are similar in several ways. Here are some common features we might want to extract:

- Extracting the center and variance. This might be the mean and standard deviation, using the `statistics` module. Or it might be the median and MAD.
- Creating a function to compute Z-scores from the mean or median.
- Applying the `filter()` function to locate outliers.

When looking at data with a large number of attributes, or looking at a large number of related data sets, it's helpful to write these functions first in the notebook. Once they've been debugged, they can be cut from the notebook and collected into a separate module. The notebook can then be modified to import the functions, making it easier to reuse these functions.

Because the source data is pushed into a dictionary with string keys, it becomes possible to consider functions that work across a sequence of key values. We might have cells that look like the following example:

```
for column in ('x', 'y'):
    values = list(
        numeric_filter(float, (s[column] for s in series_4_data))
    )
    m = median(values)
    print(column, len(series_4_data), len(values), m)
```

This will analyze all of the columns named in the surrounding `for` statement. In this example, the `x` and `y` column names are provided as the collection of columns to analyze. The result is a small table of values with the column name, the raw data size, the filtered data size, and the median of the filtered data.

The idea of a collection of descriptive statistics suggests a class to hold these. We might add the following dataclass:

```
from dataclasses import dataclass

@dataclass
class AttrSummary:
    name: str
    raw_count: int
    valid_count: int
    median: float

    @classmethod
    def from_raw(
        cls: Type["AttrSummary"],
        column: str,
        text_values: list[str]
    ) -> "AttrSummary":
        values = list(numeric_filter(float, text_values))
        return cls(
            name=column,
            raw_count=len(text_values),
```

```
        valid_count=len(values),  
        median=median(values)  
    )
```

The class definition includes a class method to build instances of this class from a collection of raw values. Putting the instance builder into the class definition makes it slightly easier to add additional inspection attributes and the functions needed to compute those attributes. A function that builds `AttrSummary` instances can be used to summarize the attributes of a data set. This function might look like the following:

```
from collections.abc import Iterator  
from typing import TypeAlias  
  
Samples: TypeAlias = list[dict[str, str]]  
  
def summary_iter(  
    samples: Samples,  
    columns: list[str]  
)-> Iterator[AttrSummary]:  
    for column in columns:  
        text = [s[column] for s in samples]  
        yield AttrSummary.from_raw(column, text)
```

This kind of function makes it possible to reuse inspection code for a number of attributes in a complicated data set. After looking at the suggested technical approach, we'll turn to the deliverables for this project.

7.1.3 Deliverables

This project has the following deliverables:

- A `requirements-dev.txt` file that identifies the tools used, usually `jupyterlab==3.5.3`.
- Documentation in the `docs` folder.
- Unit tests for any new changes to the modules in use.
- Any new application modules with code to be used by the inspection notebook.

- A notebook to inspect the attributes that appear to have cardinal data.

This project will require a `notebooks` directory. See [*List of deliverables*](#) for some more information on this structure.

We'll look at a few of these deliverables in a little more detail.

Inspection module

You are encouraged to refactor functions like `samples_iter()`, `non_numeric()`, and `numeric_filter()` into a separate module. Additionally, the `AttrSummary` class and the closely related `summary_iter()` function are also good candidates for being moved to a separate module with useful inspection classes and functions.

Notebooks can be refactored to import these classes and functions from a separate module.

It's easiest to throw this module into the `notebooks` folder to make it easier to access. An alternative is to include the `src` directory on the `PYTHONPATH` environment variable, making it available to the Jupyter Lab session.

Another alternative is to create an IPython profile with the `ipython profile create` command at the terminal prompt. This will create a `~/.ipython/profile_default` directory with the default configuration files in it. Adding a `startup` folder permits including scripts that will add the `src` directory to the `sys.path` list of places to look for modules.

See

<https://ipython.readthedocs.io/en/stable/interactive/tutorial.html#startup-files>.

Unit test cases for the module

The various functions were refactored from a notebook to create a separate module need unit tests. In many cases, the functions will have doctest examples; the notebook as a whole will have a doctest cell.

In this case, an extra option in the **pytest** command will execute these tests, as well.

```
% pytest --doctest-modules notebooks/*.py
```

The `--doctest-modules` option will look for the doctest examples and execute them.

An alternative is to use the Python `doctest` command directly.

```
% python -m doctest notebooks/*.py
```

It is, of course, essential to test the code extracted from the notebook to be sure it works properly and can be trusted.

This revised and expanded inspection notebook lets an analyst inspect unknown data sources to confirm values are likely to be cardinal numbers, for example, measures or counts. Using a filter function can help locate invalid or other anomalous text. Some statistical techniques can help to locate outlying values.

In the next project, we'll look at non-cardinal data. This includes nominal data (i.e., strings of digits that aren't numbers), and ordinal values that represent ranking or ordering positions.

7.2 Project 2.3: Validating text and codes — nominal data and ordinal numbers

7.2.1 Description

In the previous project ([***Project 2.2: Validating cardinal domains — measures, counts, and durations***](#)), we looked at attributes that contained cardinal data – measures and counts. We also need to look at ordinal and nominal data. Ordinal data is generally used to provide ranks and ordering. Nominal data is best thought of as codes made up of strings of digits. Values like US postal codes and bank account numbers are nominal data.

When we look at the **CO2 PPM — Trends in Atmospheric Carbon Dioxide** data set, available at <https://datahub.io/core/co2-ppm>, it has dates that are provided in two forms: as a `year-month-day` string and as a decimal number. The decimal number positions the first day of the month within the year as a whole.

It's instructive to use ordinal day numbers to compute unique values for each date and compare these with the supplied "Decimal Date" value. An integer day number may be more useful than the decimal date value because it avoids truncation to three decimal places.

Similarly, many of the data sets available from <https://berkeleyearth.org/data/> contain complicated date and time values. Looking at the source data, <https://berkeleyearth.org/archive/source-files/> has data sets with nominal values to encode precipitation types or other details of historical weather. For even more data, see <https://www.ncdc.noaa.gov/cdo-web/>. All of these datasets have dates in a variety of formats.

What's important for this project is to get an overview of the data that involves dates and nominal code values. Later projects will look at cleaning and converting the data for further use. This notebook is only designed to preview and inspect the data. It is used to demonstrate the data is complete and consistent and can be used for further processing.

Dates and times

A date, time, and the combined date-time value represent a specific point in time, sometimes called a timestamp. Generally, these are modeled by

Python `datetime` objects.

A date in isolation can generally be treated as a `datetime` with a time of midnight. A time in isolation is often part of a date stated elsewhere in the data or assumed from context. Ideally, a date-time value has been broken into separate columns of data for no good reason and can be combined. In other cases, the data might be a bit more difficult to track down. For example, a log file as a whole might have an implied date — because each log file starts at midnight UTC — and the time values must be combined with the (implied) log's date.

Date-time values are quite complex and rich with strange quirks. To keep the Gregorian calendar aligned with the positions of stars, and the Moon, leap days are added periodically. The `datetime` library in Python is the best way to work with the calendar.

It's generally a bad idea to do any date-time computation outside the `datetime` package.

Home-brewed date computations are difficult to implement correctly.

The `toordinal()` function of a `datetime.datetime` object provides a clear relationship between dates and an ordinal number that can be used to put dates into order.

Because months are irregular, there are several common kinds of date computations:

- A date plus or minus a duration given in months. The day of the month is generally preserved, except in the unusual case of February 29, 30, or 31, where ad hoc rules will apply.
- A date plus or minus a duration given in days or weeks.

These kinds of computations can result in dates in a different year. For month-based computations, an ordinal month value needs to be computed from the date. Given a date, d , with a year, $d.y$, and a month $d.m$, the ordinal month, m_o , is $d.y \times 12 + d.m - 1$. After a computation, the `di-`

`vmod()` function will recover the year and month of the result. Note that months are generally numbered from 1, but the ordinal month computation numbers months from zero. This leads to a `-1` when creating an ordinal month from a date, and a `+1` when creating a date from an ordinal month. As noted above, when the resulting month is February, something needs to be done to handle the exceptional case of trying to build a possibly invalid date with a day number that's invalid in February of the given year.

For day- or week-based computations, the `toordinal()` function and `fromordinal()` functions will work correctly to order and compute differences between dates.

All calendar computations must be done using ordinal values.

Here are the three steps:

1. Either use the built-in `toordinal()` method of a `datetime` object, or compute an ordinal month number.
2. Apply duration offsets to the ordinal value.
3. Either use the built-in `fromordinal()` class method of the `datetime` class, or use the `divmod()` function to compute the year and month of the ordinal month number.

For some developers, the use of ordinal numbers for dates can feel complicated. Using `if` statements to decide if an offset from a date is in a different year is less reliable and requires more extensive edge-case testing. Using an expression like `year, month = divmod(date, 12)` is much easier to test.

In the next section, we'll look at time and the problem of local time.

Time values, local time, and UTC time

Local time is subject to a great deal of complex-seeming rules, particularly in the US. Some countries have a single time zone, simplifying what constitutes local time. In the US, however, each county decides which

timezone it belongs to, leading to very complex situations that don't necessarily follow US state borders.

Some countries (the US and Europe, as well as a scattering of other places) offset the time (generally, but not universally by one hour) for part of the year. The rules are not necessarily nationwide; Canada, Mexico, Australia, and Chile have regions that don't have daylight savings time offsets. The Navajo nation — surrounded by the state of Arizona in the US — doesn't switch its clocks.

The rules are here: <https://data.iana.org/time-zones/tz-link.html>. This is part of the Python `datetime` library and is already available in Python.

This complexity makes use of the **universal coordinated time (UTC)** imperative.

Local times should be converted into UTC for analysis purposes.

See <https://www.rfc-editor.org/rfc/rfc3339> for time formats that can include a local-time offset.

UTC can be converted back into local time to be displayed to users.

7.2.2 Approach

Dates and times often have bewildering formats. This is particularly true in the US, where dates are often written as numbers in month/day/year format. Using year/month/day puts the values in order of significance. Using day/month/year is the reverse order of significance. The US ordering is simply strange.

This makes it difficult to do inspections on completely unknown data without any metadata to explain the serialization format. A date like 01/02/03 could mean almost anything.

In some cases, a survey of many date-like values will reveal a field with a range of 1-12 and another field with a range of 1-31, permitting analysts to distinguish between the month and day. The remaining field can be taken as a truncated year.

In cases where there is not enough data to make a positive identification of month or day, other clues will be needed. Ideally, there's metadata to define the date format.

The `datetime.strptime()` function can be used to parse dates when the format(s) are known. Until the date format is known, the data must be used cautiously.

Here are two Python modules that can help parse dates:

- <https://pypi.org/project/dateparser/>
- <https://pypi.org/project/python-dateutil/>

It's important to carefully inspect the results of date parsing to be sure the results are sensible. There are some confounding factors.

Years, for example, can be provided as two or four digits. For example, when dealing with old data, it's important to note the use of two-digit encoding schemes. For a few years prior to 2000, the year of date might have been given as a complicated two-digit transformation. In one scheme, values from 0 to 29 meant years 2000 to 2029. Values from 30 to 99 meant years 1930 to 1999. These rules were generally ad hoc, and different enterprises may have used different year encodings.

Additionally, leap seconds have been added to the calendar a few times as a way to keep the clocks aligned with planetary motion. Unlike leap years, these are the result of ongoing research by astronomers, and are not defined by the way leap years are defined.

See <https://www.timeanddate.com/time/leapseconds.html> for more information.

The presence of a leap second means that a timestamp like `1972-06-30T23:59:60` is valid. The 60 value for seconds represents the additional leap second. As of this book's initial publication, there were 26 leap seconds, all added on June 30 or December 31 of a given year. These values are rare but valid.

Nominal data

Nominal data is not numeric but may consist of strings of digits, leading to possible sources of confusion and — in some cases — useless data conversions. While nominal data should be treated as text, it's possible for a spreadsheet to treat US Postal ZIP codes as numbers and truncate the leading zeroes. For example, North Adams, MA, has a ZIP code of 01247. A spreadsheet might lose the leading zero, making the code 1247.

While it's generally best to treat nominal data as text, it may be necessary to reformat ZIP codes, account numbers, or part numbers to restore the leading zeroes. This can be done in a number of ways; perhaps the best is to use f-strings to pad values on the left with leading "0" characters. An expression like `f"{zip:0>5s}"` creates a string from the `zip` value using a format of `0>5s`. This format has a padding character, `0`, a padding rule of `>`, and a target size of `5`. The final character `s` is the type of data expected; in this case, a string.

An alternative is something like `(5*"0" + zip)[-5:]` to pad a given `zip` value to 5 positions. This prepends zeroes and then takes the rightmost five characters. It doesn't seem as elegant as an f-string but can be more flexible.

Extend the data inspection module

In the previous project, ***Project 2.2: Validating cardinal domains — measures, counts, and durations***, we considered adding a module with some useful functions to examine cardinal data. We can also add functions for ordinal and nominal data.

For a given problem domain, the date parsing can be defined as a separate, small function. This can help to avoid the complicated-looking `strptime()` function. In many cases, there are only a few date formats, and a parsing function can try the alternatives. It might look like this:

```
import datetime

def parse_date(source: str) -> datetime.datetime:
    formats = "%Y-%m-%d", "%y-%m-%d", "%Y-%b-%d"
    for fmt in formats:
        try:
            return datetime.datetime.strptime(source, fmt)
        except ValueError:
            pass
    raise ValueError(f"datetime data {source!r} not in any of {formats}
format")
```

This function has three date formats that it attempts to use to convert the data. If none of the formats match the data, a `ValueError` exception is raised.

For rank ordering data and codes, a notebook cell can rely on a `collections.Counter` instance to get the domain of values. More sophisticated processing is not required for simple numbers and nominal codes.

7.2.3 Deliverables

This project has the following deliverables:

- A `requirements-dev.txt` file that identifies the tools used, usually `jupyterlab==3.5.3`.
- Documentation in the `docs` folder.
- Unit tests for any new changes to the modules in use.
- Any new application modules with code to be used by the inspection notebook.
- A notebook to inspect the attributes that appear to have ordinal or nominal data.

The project directory structure suggested in [*Chapter 1, Project Zero: A Template for Other Projects*](#) mentions a `notebooks` directory. See [*List of deliverables*](#) for some more information. For this project, the notebook directory is needed.

We'll look at a few of these deliverables in a little more detail.

Revised inspection module

Functions for date conversions and cleaning up nominal data can be written in a separate module. Or they can be developed in a notebook, and then moved to the inspection module. As we noted in the [*Description*](#) section, this project's objective is to support the inspection of the data and the identification of special cases, data anomalies, and outlier values.

Later, we can look at refactoring these functions into a more formal and complete data cleansing module. This project's goal is to inspect the data and write some useful functions for the inspection process. This will create seeds to grow a more complete solution.

Unit test cases

Date parsing is — perhaps — one of the more awkwardly complicated problems. While we often think we've seen all of the source data formats, some small changes to upstream applications can lead to unexpected changes for data analysis purposes.

Every time there's a new date format, it becomes necessary to expand the unit tests with the bad data, and then adjust the parser to handle the bad data. This can lead to a surprisingly large number of date-time examples.

When confronted with a number of very similar cases, the `pytest` parameterized fixtures are very handy. These fixtures provide a number of examples of a test case.

The fixture might look like the following:

```
import pytest

EXAMPLES = [
    ('2021-01-18', datetime.datetime(2021, 1, 18, 0, 0)),
    ('21-01-18', datetime.datetime(2021, 1, 18, 0, 0)),
    ('2021-jan-18', datetime.datetime(2021, 1, 18, 0, 0)),
]

@pytest.fixture(params=EXAMPLES)
def date_example(request):
    return request.param
```

Each of the example values is a two-tuple with input text and the expected `datetime` object. This pair of values can be decomposed by the test case.

A test that uses this fixture full of examples might look like the following:

```
def test_date(date_example):
    text, expected = date_example
    assert parse_date(text) == expected
```

This kind of test structure permits us to add new formats as they are discovered. The test cases in the `EXAMPLES` variable are easy to expand with additional formats and special cases.

Now that we've looked at inspecting cardinal, ordinal, and nominal data, we can turn to a more specialized form of nominal data: key values used to follow references between separate data sets.

7.3 Project 2.4: Finding reference domains

In many cases, data is decomposed to avoid repetition. In [*Chapter 5, Data Acquisition Features: SQL Database*](#), we touched on the idea of normalization to decompose data.

As an example, consider the data sets in this directory:
<https://www.ncei.noaa.gov/pub/data/paleo/historical/northamerica/usa/new-england/>

There are three separate files. Here’s what we see when we visit the web page.

Here’s the index of the
/pub/data/paleo/historical/northamerica/usa/new-england file:

Name	Last modified	Size	Description
Parent Directory		-	
new-england-oldweather-data.txt	2014-01-30 13:02	21M	
readme-new-england-oldweather.txt	2014-01-29 19:22	9.6K	
town-summary.txt	2014-01-29 18:51	34K	

The `readme-new-england-oldweather.txt` file has descriptions of a number of codes and their meanings used in the main data set. The "readme" file provides a number of mappings from keys to values. The keys are used in the massive "oldweather-data" file to reduce the repetition of data.

These mappings include the following:

- The Temperature Code Key
- The Precipitation Type Key
- The Precipitation Amount key
- The Snowfall Amount key
- The Like Values Code key
- The Pressure Code Key
- The Sky Cover Key
- The Sky Classification Key
- The Location Code Key

This is a rather complex decomposition of primary data into coded values.

7.3.1 Description

In cases where data is decomposed or normalized, we need to confirm that references between items are valid. Relationships are often one-way — a sample will have a reference to an item in another collection of data. For example, a climate record may have a reference to "Town Id" (TWID) with a value like `NY26`. A second data set with the "location code key" provides detailed information on the definition of the `NY26` town ID. There's no reverse reference from the location code data set to all of the climate records for that location.

We often depict this relationship as an ERD. For example, ***Figure 7.1***.

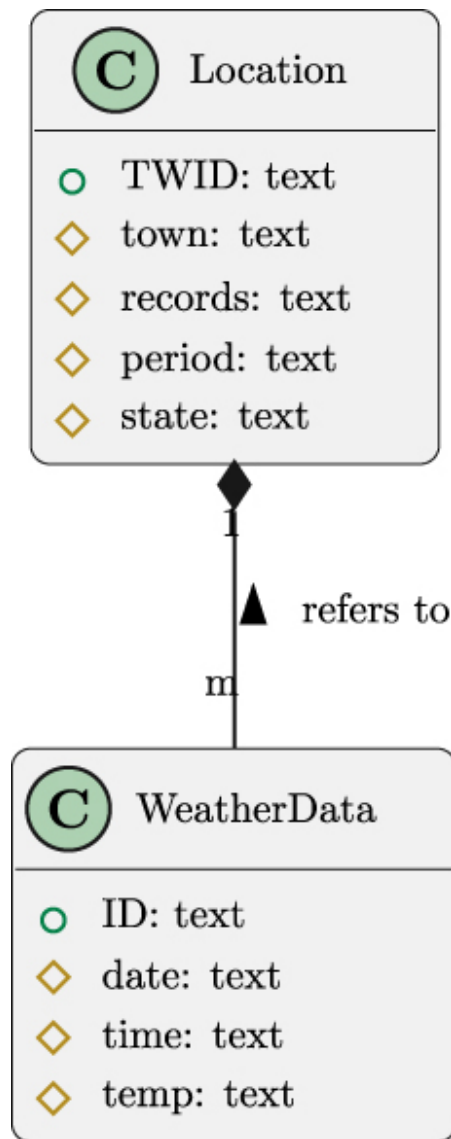


Figure 7.1: A Normalized Relationship

A number of weather data records refer to a single location definition.

The database designers will call the Location’s “TWID” attribute a **primary key**. The WeatherData’s ID attribute is called a **foreign key**; it’s a primary key for a different class of entities. These are often abbreviated as PK and FK.

There are two closely related questions about the relationship between entities:

- What is the *cardinality* of the relationship? This must be viewed from both directions. How many primary key entities have relationships

with foreign key entities? How many foreign key entities have a relationship with a primary key entity?

- What is the *optionality* of the relationship? Again, we must ask this in both directions. Must a primary entity have any foreign key references? Must the foreign key item have a primary key reference?

While a large number of combinations are possible, there are a few common patterns.

- The mandatory many-to-one relationship. This is exemplified by the historical weather data. Many weather data records must refer to a single location definition. There are two common variants. In one case, a location **must** have one or more weather records. The other common variant may have locations without any weather data that refers to the location.
- An optional one-to-one relationship. This isn't in the weather data example, but we may have invoices with payments and invoices without payments. The relationship is one-to-one, but a payment may not exist yet.
- A many-to-many relationship. An example of a many-to-many relationship is a product entity that has a number of features. Features are reused between products. This requires a separate many-to-many association table to track the relationship links.

This leads to the following two detailed inspections:

1. The domain of primary key values. For example, the “TWID” attribute of each location.
2. The domain of the foreign key values. For example, the ID attribute of each weather data record.

If these two sets are identical, we can be sure the foreign keys all have matching primary keys. We can count the number of rows that share a foreign key to work out the cardinality (and the optionality) of the relationship.

If the two sets are not identical, we have to determine which set has the extra rows. Let's call the two sets P and F . Further, we know that $P \neq F$. There are a number of scenarios:

- $P \supset F$: This means there are some primary keys without any foreign keys. If the relationship is optional, then, there's no problem. The $P \setminus F$ is the set of unused entities.
- $F \subset P$: This means there are foreign keys that do not have an associated primary key. This situation may be a misunderstanding of the key attributes, or it may mean data is missing.

What's important for this project is to have an overview of key values and their relationship. This notebook is only designed to preview and inspect the data. It is used to demonstrate the data is complete and consistent, and can be used for further processing.

In the next section, we'll look at how we can build cells in a notebook to compare the keys and determine the cardinality of the relationships.

7.3.2 Approach

To work with data sets like

<https://www.ncei.noaa.gov/pub/data/paleo/historical/northamerica/usa/new-england/> we'll need to compare keys.

This will lead to two kinds of data summarization cells in an inspection notebook:

- Summarizing primary keys in a `Counter` object.
- Summarizing foreign key references to those primary keys, also using a `Counter`.

Once the `Counter` summaries are available, then the `.keys()` method will have the distinct primary or foreign key values. This can be transformed into a Python `set` object, permitting elegant comparison, subset checking, and set subtraction operations.

We'll look at the programming to collect key values and references to keys first. Then, we'll look at summaries that are helpful.

Collect and compare keys

The core inspection tool is the `collections.Counter` class. Let's assume we have done two separate data acquisition steps. The first extracted the location definitions from the `readme-new-england-oldweather.txt` file. The second converted all of the `new-england-oldweather-data.txt` weather data records into a separate file.

The inspection notebook can load the location definitions and gather the values of the `TWID` attribute.

One cell for loading the key definitions might look like this:

```
from pathlib import Path
from inspection import samples_iter

location_path = Path("/path/to/location.ndjson")
with location_path.open() as data_file:
    locations = list(samples_iter(data_file))
```

A cell for inspecting the definitions of the town keys might look like this:

```
import collections

town_id_count = collections.Counter(
    row['TWID'] for row in locations
)
town_id_set = set(town_id_count.keys())
```

This creates the `town_id_set` variable with a set of IDs in use. The values of the `town_id_counts` variable are the number of location definitions for each ID. Since this is supposed to be a primary key, it should have only a single instance of each value.

The data with references to the town keys may be much larger than the definitions of the keys. In some cases, it's not practical to load all the data

into memory, and instead, the inspection needs to work with summaries of selected columns.

For this example, that means a `list` object is **not** created with the weather data. Instead, a generator expression is used to extract a relevant column, and this generator is then used to build the final summary `Counter` object.

The rows of data with references to the foreign keys might look like this:

```
weather_data_path = Path("/path/to/weather-data.ndjson")
with weather_data_path.open() as data_file:
    weather_reports = samples_iter(data_file)
    weather_id_count = collections.Counter(
        row['ID'] for row in weather_reports
    )
```

Once the `weather_id_count` summary has been created, the following cell can compute the domain of key references like this:

```
weather_id_set = set(weather_id_count.keys())
```

It's important to note that this example emphatically does *not* create a list of individual weather report samples. That would be a lot of data jammed into memory at one time. Instead, this example uses a generator expression to extract the `'ID'` attribute from each row. These values are used to populate the `weather_id_count` variable. This is used to extract the set of IDs in use in the weather reports.

Since we have two sets, we can use Python's set operations to compare the two. Ideally, a cell can assert that `weather_id_set == town_id_set`. If the two are not equal, then the set subtraction operation can be used to locate anomalous data.

Summarize keys counts

The first summary is the comparison of primary keys to foreign keys. If the two sets don't match, the list of missing foreign keys may be helpful

for locating the root cause of the problem.

Additionally, the range of counts for a foreign key provides some hints as to its cardinality and optionality. When a primary key has no foreign key values referring to it, the relationship appears optional. This should be confirmed by reading the metadata descriptions. The lower and upper bounds on the foreign key counts provide the range of the cardinality. Does this range make sense? Are there any hints in the metadata about the cardinality?

The example data source for this project includes a file with summary counts. The `town-summary.txt` file has four columns: “STID”, “TWID”, “YEAR”, and “Records”. The “STID” is from the location definitions; it’s the US state. The “TWID” is the town ID. The “YEAR” is from the weather data; it’s the year of the report. Finally, the “Records” attribute is the count of weather reports for a given location and year.

The Town ID and Year form a logical pair of values that can be used to build a `collections.Counter` object. To fully reproduce this table, though, the location definitions are needed to map a Town ID, “TWID,” to the associated state, “STID.”

While it’s also possible to decompose the “TWID” key to extract the state information from the first two characters, this is not a good design alternative. This composite key is an uncommon kind of key design. It’s considerably more common for primary keys to be atomic with no internal information available. A good design treats the key as an opaque identifier and looks up the state information in the associated location definition table from the `readme` file.

7.3.3 Deliverables

This project has the following deliverables:

- A `requirements-dev.txt` file that identifies the tools used, usually `jupyterlab==3.5.3`.

- Documentation in the `docs` folder.
- Unit tests for any new changes to the modules in use.
- Any new application modules with code to be used by the inspection notebook.
- A notebook to inspect the attributes that appear to have foreign or primary keys.

The project directory structure suggested in [*Chapter 1, Project Zero: A Template for Other Projects*](#) mentions a `notebooks` directory. See [*List of deliverables*](#) for some more information. For this project, the notebook directory is needed.

We'll look at a few of these deliverables in a little more detail.

Revised inspection module

The functions for examining primary and foreign keys can be written in a separate module. It's often easiest to develop these in a notebook first. There can be odd discrepancies that arise because of misunderstandings. Once the key examination works, it can be moved to the inspection module. As we noted in the [*Description*](#), this project's objective is to support the inspection of the data and the identification of special cases, data anomalies, and outlier values.

Unit test cases

It's often helpful to create test cases for the most common varieties of key problems: primary keys with no foreign keys and foreign keys with no primary keys. These complications don't often arise with readily available, well-curated data sets; they often arise with enterprise data with incomplete documentation.

This can lead to rather lengthy fixtures that contain two collections of source objects. It doesn't take many rows of data to reveal a missing key; two rows of data are enough to show a key that's present and a row with a missing key.

It's also essential to keep these test cases separate from test cases for cardinal data processing, and ordinal data conversions. Since keys are a kind of nominal data, a key cardinality check may be dependent on a separate function to clean damaged key values.

For example, real data may require a step to add leading zeroes to account numbers before they can be checked against a list of transactions to find transactions for the account. These two operations on account number keys need to be built — and tested — in isolation. The data cleanup application can combine the two functions. For now, they are separate concerns with separate test cases.

Revised notebook to use the refactored inspection model

A failure to resolve foreign keys is a chronic problem in data acquisition applications. This is often due to a wide variety of circumstances, and there's no single process for data inspection. This means a notebook can have a spectrum of information in it. We might see any of the following kinds of cells:

- A cell explaining the sets of keys match, and the data is likely usable.
- A cell explaining some primary keys have no foreign key data. This may include a summary of this subset of samples, separate from samples that have foreign key references.
- A cell explaining some foreign keys that have no primary key. These may may reflect errors in the data. It may reflect a more complex relationship between keys. It may reflect a more complicated data model. It may reflect missing data.

In all cases, an extra cell with some markdown explaining the results is necessary. In the future, you will be grateful because in the past, you left an explanation of an anomaly in your notebook.

7.4 Summary

This chapter expanded on the core features of the inspection notebook. We looked at handling cardinal data (measures and counts), ordinal data (dates and ranks), and nominal data (codes like account numbers).

Our primary objective was to get a complete view of the data, prior to formalizing our analysis pipeline. A secondary objective was to leave notes for ourselves on outliers, anomalies, data formatting problems and other complications. A pleasant consequence of this effort is to be able to write some functions that can be used downstream to clean and normalize the data we've found.

Starting in ***Chapter 9, Project 3.1: Data Cleaning Base Application***, we'll look at refactoring these inspection functions to create a complete and automated data cleaning and normalization application. That application will be based on the lessons learned while creating inspection notebooks.

In the next chapter, we'll look at one more lesson that's often learned from the initial inspection. We often discover the underlying schema behind multiple, diverse sources of data. We'll look at formalizing the schema definition via JSONSchema, and using the schema to validate data.

7.5 Extras

Here are some ideas for you to add to the projects in this chapter.

7.5.1 Markdown cells with dates and data source information

A minor feature of an inspection notebook is some identification of the date, time, and source of the data. It's sometimes clear from the context what the data source is; there may, for example, be an obvious path to the data.

However, in many cases, it's not perfectly clear what file is being inspected or how it was acquired. As a general solution, any processing application should produce a log. In some cases, a metadata file can include the details of the processing steps.

This additional metadata on the source and processing steps can be helpful when reviewing a data inspection notebook or sharing a preliminary inspection of data with others. In many cases, this extra data is pasted into ordinary markdown cells. In other cases, this data may be the result of scanning a log file for key `INFO` lines that summarize processing.

7.5.2 Presentation materials

A common request is to tailor a presentation to users or peers to explain a new source of data, or explain anomalies found in existing data sources. These presentations often involve an online meeting or in-person meeting with some kind of “slide deck” that emphasizes the speaker's points.

Proprietary tools like Keynote or PowerPoint are common for these slide decks.

A better choice is to organize a notebook carefully and export it as `reveal.js` slides.

The RISE extension for Jupyter is popular for this. See

<https://rise.readthedocs.io/en/stable/>.

Having a notebook that is **also** the slide presentation for business owners and users provide a great deal of flexibility. Rather than copying and pasting to move data from an inspection notebook to PowerPoint (or Keynote), we only need to make sure each slide has a few key points about the data. If the slide has a data sample, it's only a few rows, which provide supporting evidence for the speaker's remarks.

In many enterprises, these presentations are shared widely. It can be beneficial to make sure the data in the presentation comes directly from the source and is immune to copy-paste errors and omissions.

7.5.3 JupyterBook or Quarto for even more sophisticated output

In some cases, a preliminary inspection of data may involve learning a lot of lessons about the data sources, encoding schemes, missing data, and relationships between data sets. This information often needs to be organized and published.

There are a number of ways to disseminate lessons learned about data:

- Share the notebooks. For some communities of users, the interactive nature of a notebook invites further exploration.
- Export the notebook for publication. One choice is to create a PDF that can be shared. Another choice is to create RST, Markdown, or LaTeX and use a publishing pipeline to build a final, shareable document.
- Use a tool like Jupyter{Book} to formalize the publication of a shareable document.
- Use Quarto to publish a final, shareable document.

For Jupyter{Book}, see <https://jupyterbook.org/en/stable/intro.html>.

The larger “Executable{Books}” project (<https://executablebooks.org/en/latest/tools.html>) describes the collection of Python-related tools, including Myst-NB, Sphinx, and some related Sphinx themes. The essential ingredient is using Sphinx to control the final publication.

For Quarto, see <https://quarto.org>. This is somewhat more tightly integrated: it requires a single download of the Quarto CLI. The Quarto tool leverages Pandoc to produce a final, elegant, ready-to-publish file.

You are encouraged to look at ways to elevate the shared notebook to an elegant report that can be widely shared.